

MAJOR PROJECT - 1

A LIGHT WEIGHT VECTOR DATABASE

PROJECT NO. : GS8

PROJECT GUIDE:-

Dr. Gaurav Saxena

PROJECT MEMBERS:-

Aman Singh Rawat(221B056)

Aniruddha Panchal(221B068)

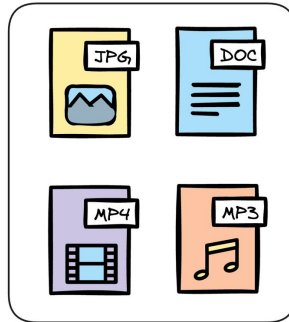
Mohnish Sadiza(221B241)

INTRODUCTION

- **What is Vector Database**

- AI/ML Integration
- Similarity Search
- Scalability
- Real-Time Performance
- Recommendation Systems
- Semantic Search

Vector Database



Unstructured data

0.2	-1.7	...	2.3
0.4	0.5	...	-1.7
4.1	-1.9	...	-1.5
-1.1	0.7	...	5.3
-3.5	2.3	...	0.5
-1.7	0.4	...	0.2
2.3	0.2	...	0.7
-1.9	4.1	...	-2.4
0.5	-1.5	...	2.3

Embeddings

RELATED WORK

- **Existing Solution:**

- Pinecone
- Annoy (Approximate Nearest Neighbors Oh Yeah)
- ScaNN (Google)
- FAISS (Facebook AI Similarity Search)



spotify/annoy

Approximate Nearest Neighbors in C++/Python
optimized for memory usage and loading/saving to
disk



65
Contributors

5k
Used by

14k
Stars

1k
Forks



OBJECTIVE AND SCOPE

- **Objective:**

- Develop a Functional Vector Database
- Implement Similarity Search
- Storage Efficiency
- Query Performance

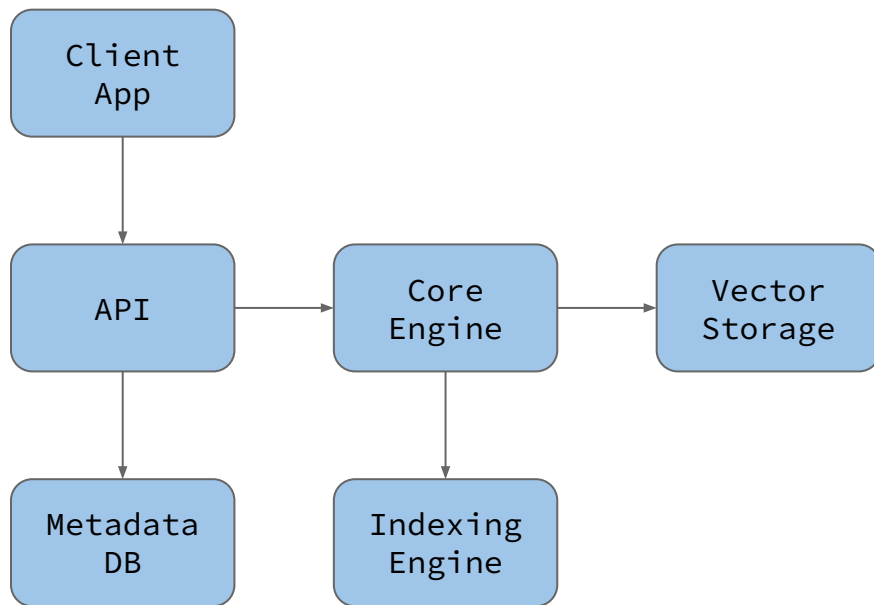
- **Scope:**

- Vector storage and retrieval operations
- Basic indexing for improved search performance
- API for database operations
- Basic metadata support for vectors

DESIGN AND IMPLEMENTATION

- **Component Description:**

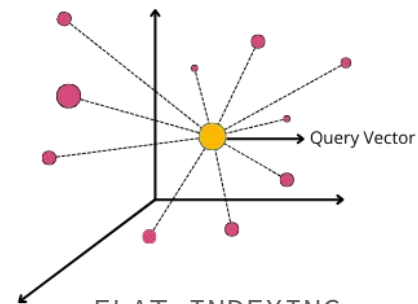
- API
- Core Engine
- Vector Storage
- Metadata Database



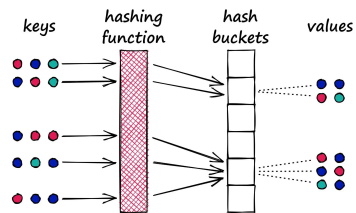
DESIGN AND IMPLEMENTATION

- **Traditional Indexing Techniques:**

- Indexing Selection Strategy
- Indexing Methods
 - Flat Index
 - KD-Tree
 - Locality Sensitive Hashing (LSH)
 - Product Quantization (PQ)
- Dynamic Index Management



FLAT INDEXING



LOCALITY SENSITIVE HASHING

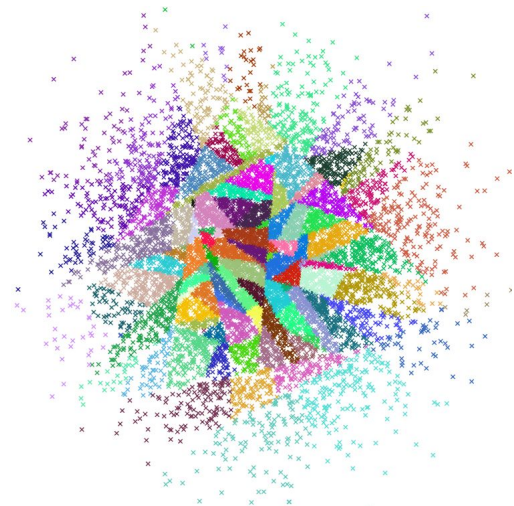
DESIGN AND IMPLEMENTATION

Index Type	Build Time	Query Time	Memory Usage	Accuracy	Best For
Flat	$O(1)$	$O(n)$	High	100%	Small datasets, exact search
KD-Tree	$O(n \log n)$	$O(\log n)$	Medium	100%	Low dimensions, exact search
LSH	$O(n)$	$O(1)^*$	High	~90%	High dimensions, flat queries
PQ	$O(n \cdot \text{iter})$	$O(m \cdot k)$	Low	~85%	Memory constrained, large data

DESIGN AND IMPLEMENTATION

- **Core Algorithms:**

- Similarity Search Algorithm
- Distance Metrics Implementation
 - Euclidean Distance
 - Cosine Similarity
 - Manhattan Distance

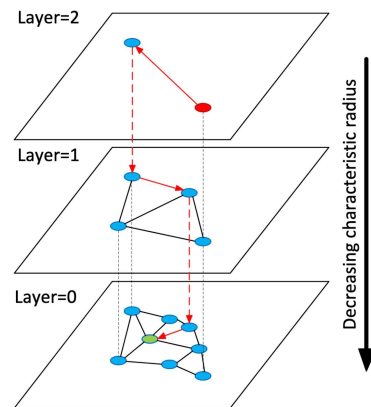


SPOTIFY (ANNOY)

DESIGN AND IMPLEMENTATION

- **HNSW (Hierarchical Navigable Small Worlds):**

- Node construction and insertion
- Multi-layer search algorithm
- Neighbor selection heuristics
- Connection pruning
- Performance Characteristics



CONCLUSION

- Our lightweight vector database project will successfully deliver a functional, efficient, and user-friendly solution for vector storage and similarity search.
- The implementation will demonstrate that complex vector database functionality can be achieved with a relatively simple architecture while maintaining good performance

REFERENCES

- Malkov, Y. A., & Yashunin, D. A. (2018). "Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs." *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4), 824-836.
- Wang, J., Liu, W., Kumar, S., & Chang, S. F. (2016). "Learning to hash for indexing big data—a survey." *Proceedings of the IEEE*, 104(1), 34-57
- Babenko, A., & Lempitsky, V. (2016). "Efficient indexing of billion-scale datasets of deep descriptors." *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2055-2063

THANK YOU